ACTIVIDAD SPRING BOOT

API REST con Spring Boot – Productos y Usuarios

Informe técnico de evidencias

Aprendiz:
Frederick Alexandro Abril Mogollón

Programa:
Tecnología en Desarrollo de Software

16 de octubre de 2026

Tabla de contenido

1. Introducción 3
2. Diagnóstico inicial 4
3. Ejecución de la API 6
4. Implementación de Lombok 8
Entidad: Product → Producto 8
Servicio: ProductService → ProductoService 9
Controlador: ProductController → ProductoController 9
5. CRUD de Productos 10
Modelo 10
Repositorio y excepción 10
Servicio 11
Controlador 11
6. CRUD de Usuarios 12
Modelo 12
Repositorio 12
Servicio 13
Controlador 13
7. Documentación 14
8. Pruebas 16
Evidencias del módulo de productos 17
Evidencias del módulo de usuarios 23
9. Conclusiones 31
10. Anexos 32

1. Introducción

El propósito de esta actividad fue terminar el proyecto Spring Boot que trabajamos en clase. Para eso tuve que encontrar y corregir los errores que impedían compilarlo y ejecutarlo, completar el CRUD del módulo de productos, construir el módulo de usuarios con la misma arquitectura, aplicar Lombok en todas las clases, documentar el proyecto y probar cada endpoint dejando evidencia del proceso. El proyecto se llama demo y conserva el paquete com.example.api.demo con el que venía.

Las tecnologías que usé fueron Java 25 con Spring Boot 4.1.1 (Spring Web MVC, Spring Data JPA y Validation), Lombok, Apache Maven con su wrapper mvnw, PostgreSQL 17 en un contenedor de Docker, JUnit 5 con MockMvc y H2 para las pruebas automáticas, Postman para las pruebas manuales, IntelliJ IDEA como entorno de desarrollo y Git con GitHub para el control de versiones.

Nota sobre la base de datos: la guía propone MySQL, pero yo trabajé con PostgreSQL porque era la base de datos que ya tenía configurada el proyecto base, con su archivo docker-compose.yml. JPA funciona igual con las dos; solo cambian el driver y la URL de conexión.

2. Diagnóstico inicial

Antes de cambiar el código hice una copia completa del proyecto original en la carpeta demo_respaldo_original. Después ejecuté mvn clean install y la aplicación, registré los errores y los corregí uno por uno. La Tabla 1 resume los errores encontrados.

Tabla 1. *Errores encontrados en el diagnóstico*

N.º	Mensaje de error	Archivo o clase	Causa	Solución aplicada	Fig.
1	FATAL: la autentificación password falló para el usuario «tiendaya» / Unable to determine Dialect without JDBC metadata	application.properties, docker-compose.yml	El puerto 5432 ya lo ocupa el PostgreSQL instalado en Windows; la aplicación se conectaba a ese servidor y no al contenedor	Publiqué el contenedor en el puerto 5433 y cambié la URL; la conexión se puede cambiar con DB_URL, DB_USER y DB_PASSWORD	1 y 2
2	DemoApplicationTests.contextLoads: Failed to load ApplicationContext (BUILD FAILURE)	DemoApplicationTests.java	La prueba levanta todo el contexto de Spring y necesita una base de datos disponible	Agregué H2 en memoria solo para pruebas, con su propio application.properties en src/test/resources	1
3	La ruta /api/productos no existía (respuesta 404)	ProductController.java	El controlador usaba /api/products y recibía los datos con @RequestParam en la URL	Lo reescribí como ProductoController en /api/productos con @RequestBody	2

El primer error fue el más difícil de entender. El mensaje decía que la contraseña del usuario tiendaya era incorrecta, pero estaba bien escrita. Al revisar los puertos ocupados vi que en mi computador ya había un PostgreSQL instalado escuchando en el 5432, así que la aplicación nunca llegaba al contenedor de Docker. Lo resolví publicando el contenedor en el puerto 5433 y apuntando la URL a ese puerto.

El segundo error era consecuencia del primero: la única prueba del proyecto, contextLoads, intentaba conectarse a la base de datos y fallaba, de modo que mvn clean install terminaba en BUILD FAILURE. Para que las pruebas no dependan de que Docker esté encendido agregué H2 con alcance de pruebas.

El tercer problema no impedía compilar, pero la API no cumplía lo que pide la guía: el controlador estaba en inglés, usaba la ruta /api/products y esperaba el nombre y el precio como parámetros de la URL en lugar de un JSON.

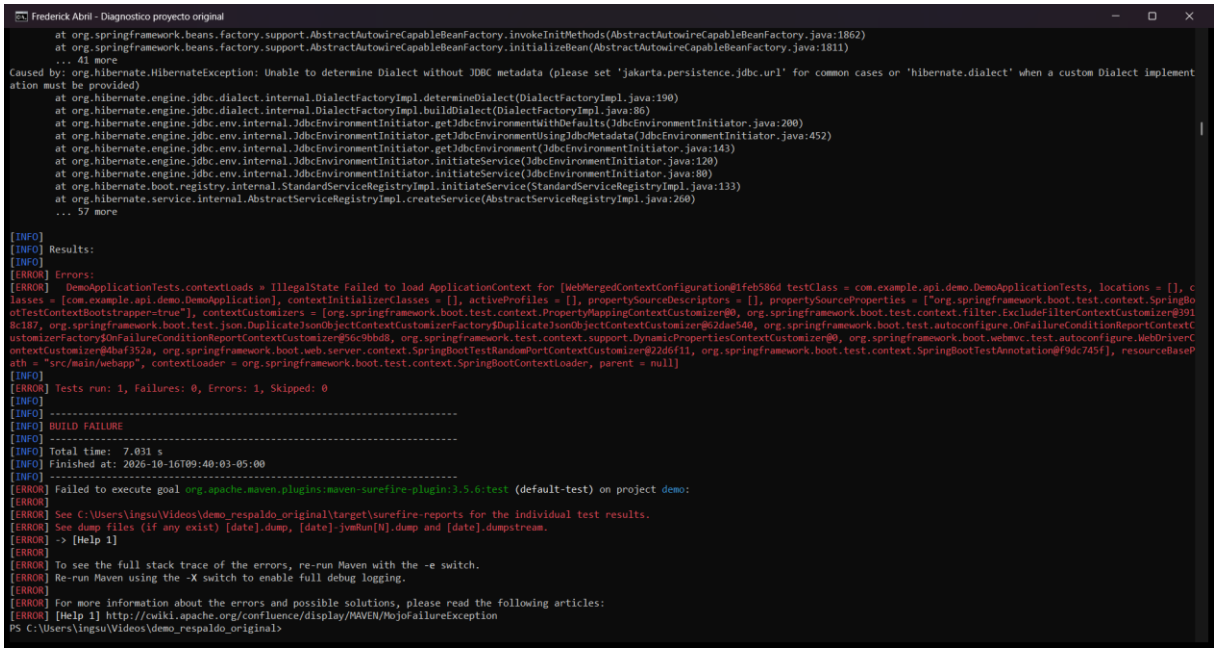


Figura 1. *Ejecución de mvn clean install sobre el proyecto original sin corregir.*

Se ve el error Unable to determine Dialect, la prueba contextLoads con un error y el resultado BUILD FAILURE. Esta captura la tomé sobre la copia de respaldo del proyecto original.

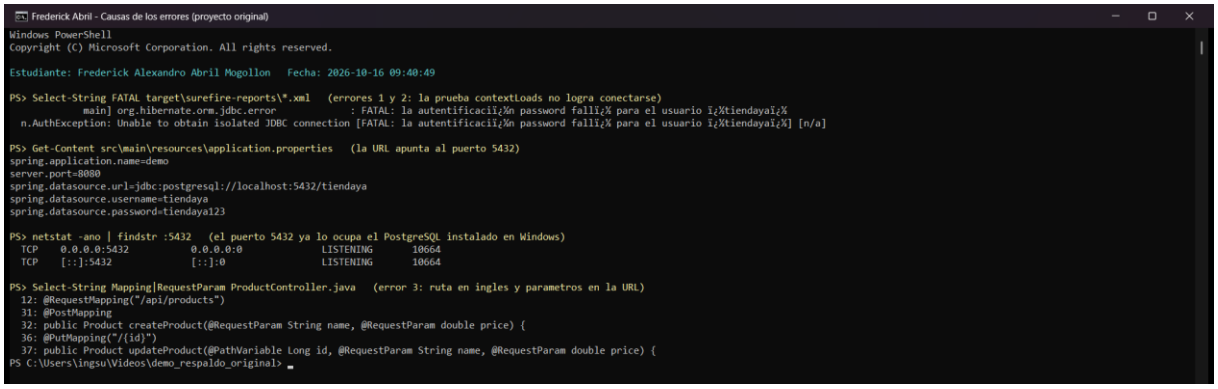


Figura 2. *Causas de los errores en el proyecto original.*

El reporte de la prueba muestra el error FATAL de autenticación (las tildes salen mal porque el PostgreSQL de Windows envía el mensaje en otra codificación), la URL apunta al puerto 5432, que ya está ocupado, y el controlador usa /api/products con @RequestParam.

3. Ejecución de la API

Después de corregir los errores, el proyecto compiló y pasó las pruebas con mvn clean install, y la aplicación arrancó conectada al contenedor tiendaya-db. En el proyecto original los cinco endpoints de productos ya existían en /api/products, pero no pude probarlos en ese estado porque la aplicación no lograba conectarse a la base de datos. Por eso los probé después de corregirlos y llevarlos a /api/productos; sus evidencias están en la sección de pruebas.

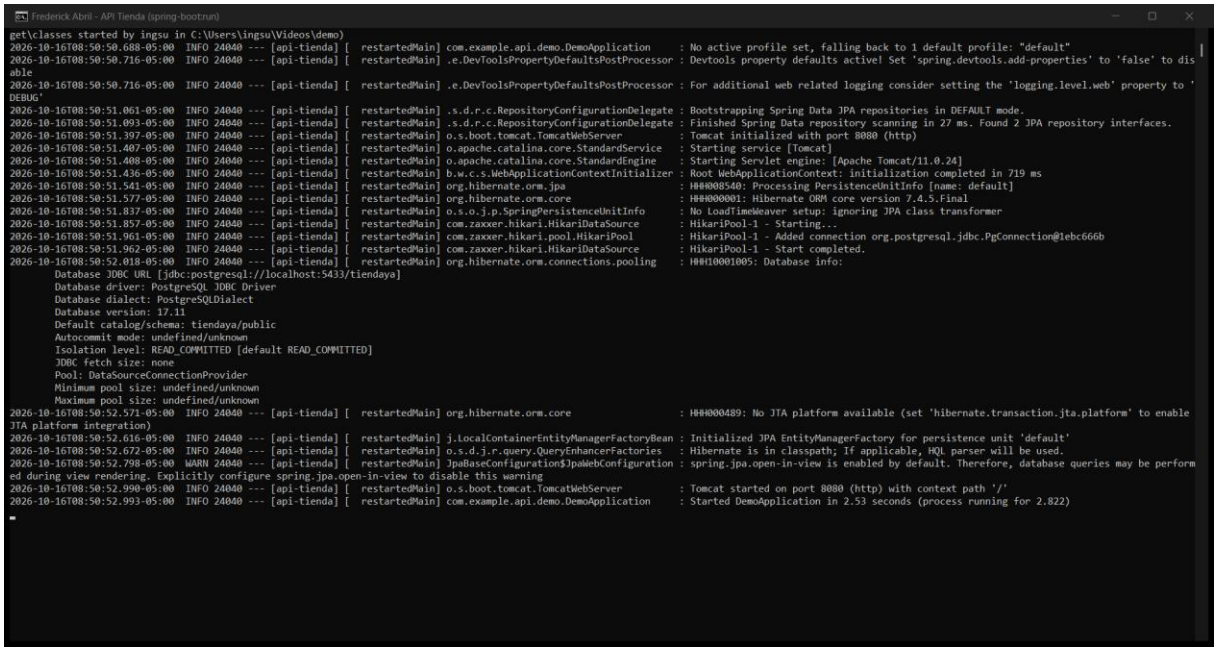


Figura 3. *Arranque de la API con mvnw spring-boot:run.*

La consola muestra la conexión a PostgreSQL en localhost:5433, Tomcat iniciado en el puerto 8080 y el mensaje Started DemoApplication, con la fecha y la hora del sistema en cada línea.

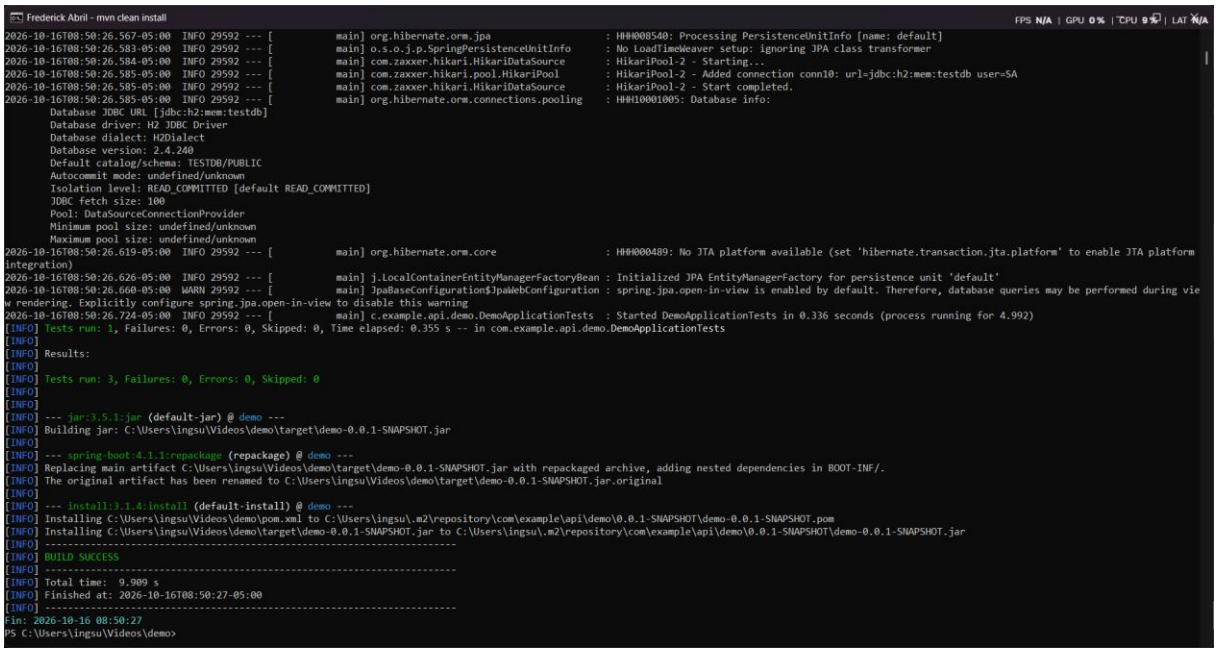


Figura 4. *mvn clean install sobre el proyecto corregido.*

Las tres pruebas automáticas pasan sin fallos (Tests run: 3, Failures: 0, Errors: 0) y el resultado es BUILD SUCCESS.

4. Implementación de Lombok

Apliqué Lombok siguiendo las anotaciones que pide la guía para cada capa. Al terminar revisé que ninguna clase tuviera getters, setters, constructores ni métodos toString escritos a mano, y que no quedara ningún @Autowired sobre atributos. La Tabla 2 muestra cada clase que intervine.

Tabla 2. *Clases intervenidas con Lombok*

Clase original	Clase actual	Antes	Después
Product	Producto	@Getter, @NoArgsConstructor, @AllArgsConstructor (sin setters)	@Getter, @Setter, @NoArgsConstructor, @AllArgsConstructor, @Builder
—	Usuario	No existía	@Getter, @Setter, @NoArgsConstructor, @AllArgsConstructor, @Builder
ProductService	ProductoService	Constructor de inyección escrito a mano	@Service, @RequiredArgsConstructor, @Slf4j
—	UsuarioService	No existía	@Service, @RequiredArgsConstructor, @Slf4j
ProductController	ProductoController	Constructor de inyección escrito a mano	@RestController, @RequiredArgsConstructor
—	UsuarioController	No existía	@RestController, @RequiredArgsConstructor
ProductResponse (record)	Se eliminó	Copiaba los datos de Product para la respuesta	Ya no hace falta: la entidad se devuelve directamente

Entidad: Product → Producto

```
// Antes (respaldo original)
@NoArgsConstructor
@AllArgsConstructor
@Getter
@Entity
@Table(name = "products")
public class Product {
```

```
// Después
@Entity
@Table(name = "productos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Producto {
```

La clase original no tenía setters, así que para actualizar había que crear un objeto nuevo con todos los datos. Con @Setter y @Builder ahora se pueden cambiar solo los campos necesarios.

Servicio: ProductService → ProductoService

```
// Antes (respaldo original)
@Service
public class ProductService {
    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
```

```
// Después
@Service
@RequiredArgsConstructor
@Slf4j
public class ProductoService {
    private final ProductoRepository productoRepository;
```

Controlador: ProductController → ProductoController

```
// Antes (respaldo original)
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }
```

```
// Después
@RestController
@RequestMapping("/api/productos")
@RequiredArgsConstructor
public class ProductoController {
    private final ProductoService productoService;
```

En el servicio y en el controlador el constructor desapareció porque @RequiredArgsConstructor lo genera con todos los atributos final, y Spring lo usa para inyectar las dependencias sin @Autowired.

5. CRUD de Productos

El proyecto original ya tenía los cinco endpoints de productos, pero en la ruta /api/products y recibiendo los datos por parámetros de la URL. Los reescribí todos en /api/productos, con cuerpo JSON, validaciones y los códigos HTTP que pide la guía, como se resume en la Tabla 3.

Tabla 3. *Endpoints del módulo de productos*

Método	Ruta	Operación	Códigos	Estado
GET	/api/productos	Listar todos	200	Existía en /api/products; corregido
GET	/api/productos/{id}	Consultar por id	200 / 404	Existía en /api/products; corregido
POST	/api/productos	Crear	201 / 400	Existía con @RequestParam; reescrito con JSON
PUT	/api/productos/{id}	Actualizar	200 / 404	Existía con @RequestParam; reescrito con JSON
DELETE	/api/productos/{id}	Eliminar	204 / 404	Existía en /api/products; corregido

Modelo

```
@Entity
@Table(name = "productos")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Producto {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank @Size(max = 100) @Column(nullable = false, length = 100)
    private String nombre;
    private String descripcion;
    @NotNull @PositiveOrZero @Column(nullable = false)
    private Double precio;
    @PositiveOrZero
    private Integer stock;
}
```

Repositorio y excepción

```
public interface ProductoRepository extends JpaRepository<Producto, Long> { }

@ResponseStatus(HttpStatus.NOT_FOUND)
public class RecursoNoEncontradoException extends RuntimeException {
    public RecursoNoEncontradoException(String mensaje) { super(mensaje); }
}
```

Servicio

```
public Producto buscarPorId(Long id) {
    return productoRepository.findById(id)
        .orElseThrow(() -> new RecursoNoEncontradoException("Producto no
encontrado con id " + id));
}
public Producto actualizar(Long id, Producto datos) {
    Producto existente = buscarPorId(id);
    existente.setNombre(datos.getNombre());
    existente.setDescripcion(datos.getDescripcion());
    existente.setPrecio(datos.getPrecio());
    existente.setStock(datos.getStock());
    return productoRepository.save(existente);
}
public void eliminar(Long id) {
    productoRepository.delete(buscarPorId(id));
}
```

Controlador

```
@PostMapping
public ResponseEntity<Producto> crear(@Valid @RequestBody Producto producto) {
    return
ResponseEntity.status(HttpStatus.CREATED).body(productoService.crear(producto));
}
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    productoService.eliminar(id);
    return ResponseEntity.noContent().build();
}
```

Los errores de los dos módulos los atiende la clase ManejadorGlobalExcepciones, anotada con @RestControllerAdvice. Devuelve 404 para RecursoNoEncontradoException y 400 para IllegalArgumentException, para los datos que no pasan las validaciones y para un JSON mal formado, siempre con un cuerpo como {"error": "mensaje"}.

6. CRUD de Usuarios

El módulo de usuarios lo construí desde cero con la misma arquitectura del de productos y con las dos reglas de negocio de la guía: el correo debe ser único y la contraseña nunca se devuelve en las respuestas.

Tabla 4. *Endpoints del módulo de usuarios*

Método	Ruta	Operación	Códigos	Estado
GET	/api/usuarios	Listar todos	200	Creado
GET	/api/usuarios/{id}	Consultar por id	200 / 404	Creado
POST	/api/usuarios	Crear	201 / 400	Creado (valida correo único)
PUT	/api/usuarios/{id}	Actualizar	200 / 400 / 404	Creado (valida correo único)
DELETE	/api/usuarios/{id}	Eliminar	204 / 404	Creado

Modelo

```
@Entity
@Table(name = "usuarios")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank @Column(nullable = false, length = 80)
    private String nombre;
    @NotBlank @Email @Column(nullable = false, unique = true, length = 120)
    private String correo;
    @NotBlank @Column(nullable = false)
    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    private String contrasena;
    private String rol;
}
```

Repositorio

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    boolean existsByCorreo(String correo);
    boolean existsByCorreoAndIdNot(String correo, Long id);
}
```

Servicio

```
public Usuario crear(Usuario usuario) {
    if (usuarioRepository.existsByCorreo(usuario.getCorreo())) {
        throw new IllegalArgumentException("Ya existe un usuario con el correo " +
usuario.getCorreo());
    }
    log.info("Creando usuario: {}", usuario.getCorreo());
    return usuarioRepository.save(usuario);
}
public Usuario actualizar(Long id, Usuario datos) {
    Usuario existente = buscarPorId(id);
    if (usuarioRepository.existsByCorreoAndIdNot(datos.getCorreo(), id)) {
        throw new IllegalArgumentException("Ya existe un usuario con el correo " +
datos.getCorreo());
    }
    existente.setNombre(datos.getNombre());
    existente.setCorreo(datos.getCorreo());
    existente.setContrasena(datos.getContrasena());
    existente.setRol(datos.getRol());
    return usuarioRepository.save(existente);
}
```

Controlador

```
@RestController
@RequestMapping("/api/usuarios")
@RequiredArgsConstructor
public class UsuarioController {
    private final UsuarioService usuarioService;

    @PostMapping
    public ResponseEntity<Usuario> crear(@Valid @RequestBody Usuario usuario) {
        return
ResponseEntity.status(HttpStatus.CREATED).body(usuarioService.crear(usuario));
    }
    // listar, obtener, actualizar y eliminar siguen el mismo patrón del módulo de
productos
}
```

Para la actualización agregué existsByCorreoAndIdNot. Así un usuario puede conservar su propio correo al actualizarse, pero no tomar el correo de otro usuario.

7. Documentación

En la raíz del proyecto está el archivo README.md, con el nombre y la descripción del proyecto, las tecnologías y sus versiones, las instrucciones para configurar la base de datos y ejecutar la aplicación, la tabla de endpoints con ejemplos de petición y respuesta, y el autor. Además, cada clase y cada método público tiene su comentario Javadoc con el propósito, los parámetros y el valor de retorno.

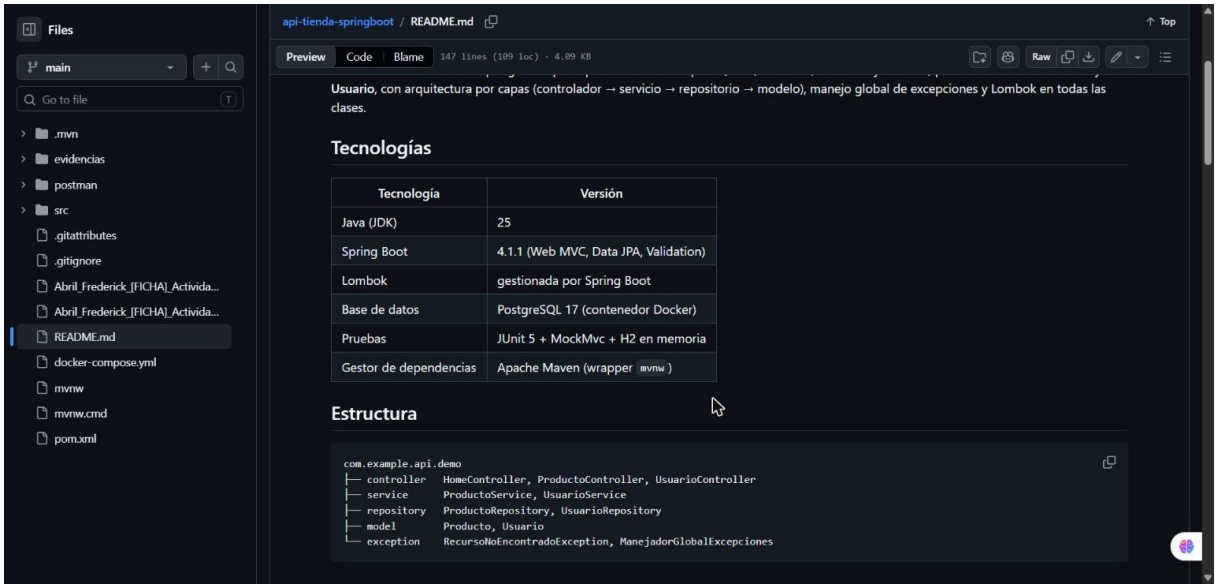


Figura 5. *Archivo README.md publicado en el repositorio de GitHub.*

Se ven las secciones de descripción, tecnologías con sus versiones y la estructura de paquetes; más abajo el archivo continúa con la configuración, la ejecución, los endpoints y el autor.

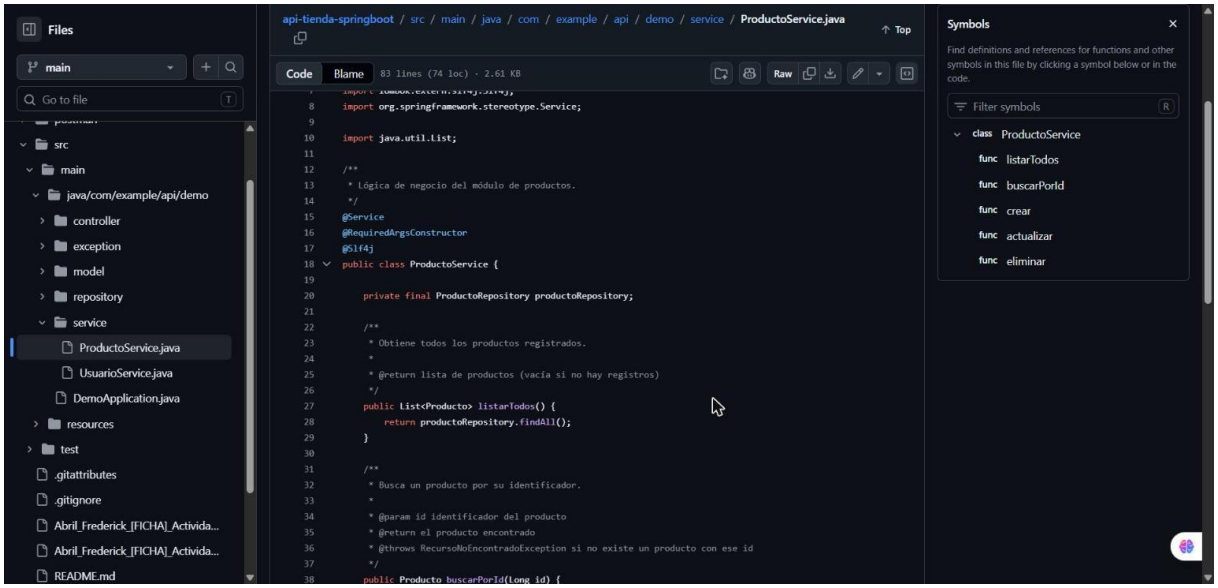


Figura 6. *Comentarios Javadoc en la clase ProductoService.*

La clase y cada método público tienen su descripción, con @param, @return y @throws cuando aplica. El resto de clases del proyecto siguen el mismo formato.

8. Pruebas

Ejecuté los casos de la Tabla 3 de la guía en Postman, con la colección API Tienda organizada en las carpetas Productos y Usuarios, el Desktop Agent conectado a localhost:8080 y el environment «Frederick Abril Mogollón», que define la variable estudiante con mi nombre y baseUrl con http://localhost:8080. Antes de empezar dejé las tablas vacías para que los identificadores coincidieran con los de la guía. Los casos 11 y 12 los hice desde la terminal y con un reinicio de la API. Además, la clase ApiCrudTests recorre la misma matriz con MockMvc cada vez que se ejecuta mvn clean install.

Tabla 5. *Matriz de casos de prueba – Productos*

N.º	Caso de prueba	Método y ruta	Esperado	Obtenido	Cumple	Fig.
1	Crear registro válido	POST /api/productos	201 y objeto con id	201, id 1	Sí	7
2	Crear segundo registro	POST /api/productos	201	201, id 2	Sí	8
3	Listar todos	GET /api/productos	200 y dos elementos	200, dos elementos	Sí	9
4	Consultar id existente	GET /api/productos/1	200 y objeto	200	Sí	10
5	Consultar id inexistente	GET /api/productos/999	404 y mensaje	404 y mensaje	Sí	11
6	Actualizar existente	PUT /api/productos/1	200 y datos nuevos	200, datos nuevos	Sí	12
7	Actualizar inexistente	PUT /api/productos/999	404	404	Sí	13
8	Eliminar existente	DELETE /api/productos/2	204	204	Sí	14
9	Eliminar inexistente	DELETE /api/productos/999	404	404	Sí	15
10	Listar después de eliminar	GET /api/productos	200 y un elemento	200, un elemento	Sí	16
11	Verificación en base de datos	SELECT * FROM productos;	Coincide con la API	Coincide	Sí	17
12	Persistencia de datos	Reiniciar API y GET /api/productos	Registros se conservan	Se conservan	Sí	18

Tabla 6. *Matriz de casos de prueba – Usuarios*

N.º	Caso de prueba	Método y ruta	Esperado	Obtenido	Cumple	Fig.
1	Crear registro válido	POST /api/usuarios	201 y objeto con id	201, id 1	Sí	19
2	Crear segundo registro	POST /api/usuarios	201	201, id 2	Sí	20
3	Listar todos	GET /api/usuarios	200 y dos elementos	200, dos elementos	Sí	21
4	Consultar id existente	GET /api/usuarios/1	200 y objeto	200	Sí	22
5	Consultar id inexistente	GET /api/usuarios/999	404 y mensaje	404 y mensaje	Sí	23
6	Actualizar existente	PUT /api/usuarios/1	200 y datos nuevos	200, datos nuevos	Sí	24
7	Actualizar inexistente	PUT /api/usuarios/999	404	404	Sí	25
8	Eliminar existente	DELETE /api/usuarios/2	204	204	Sí	26
9	Eliminar inexistente	DELETE /api/usuarios/999	404	404	Sí	27
10	Listar después de eliminar	GET /api/usuarios	200 y un elemento	200, un elemento	Sí	28
11	Verificación en base de datos	SELECT * FROM usuarios;	Coincide con la API	Coincide	Sí	29
12	Persistencia de datos	Reiniciar API y GET /api/usuarios	Registros se conservan	Se conservan	Sí	30
13	Correo duplicado	POST /api/usuarios	400 y mensaje	400 y mensaje	Sí	31
14	Contraseña oculta	GET /api/usuarios/1	Sin campo contrasena	Sin campo contrasena	Sí	32

Evidencias del módulo de productos

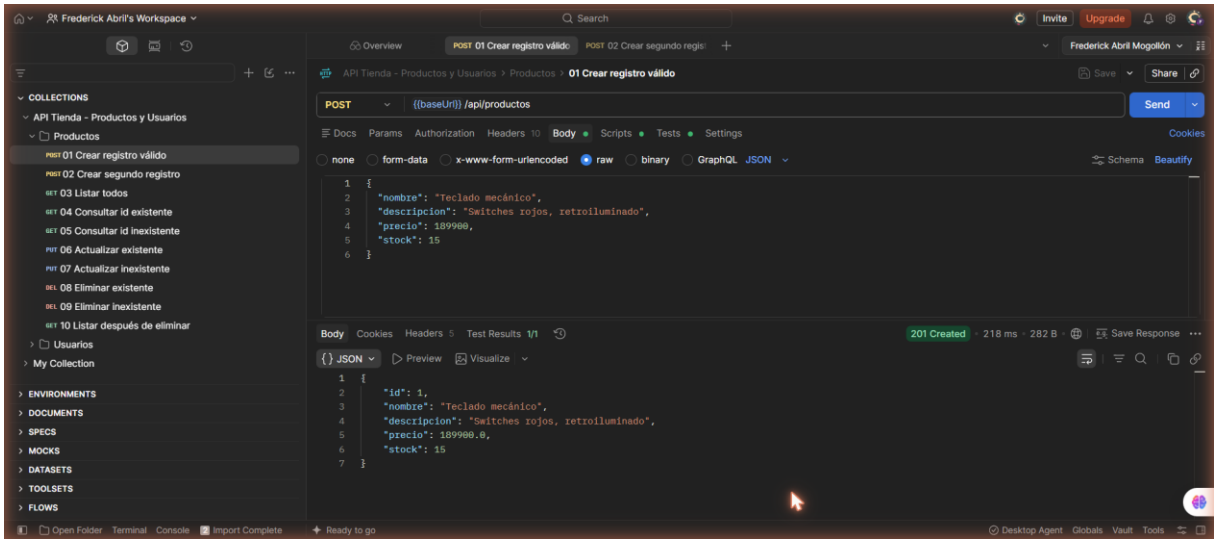


Figura 7. *Caso 1 – Crear producto válido.*

POST /api/productos con el teclado mecánico. La API responde 201 Created y devuelve el producto con id 1 asignado por la base de datos.

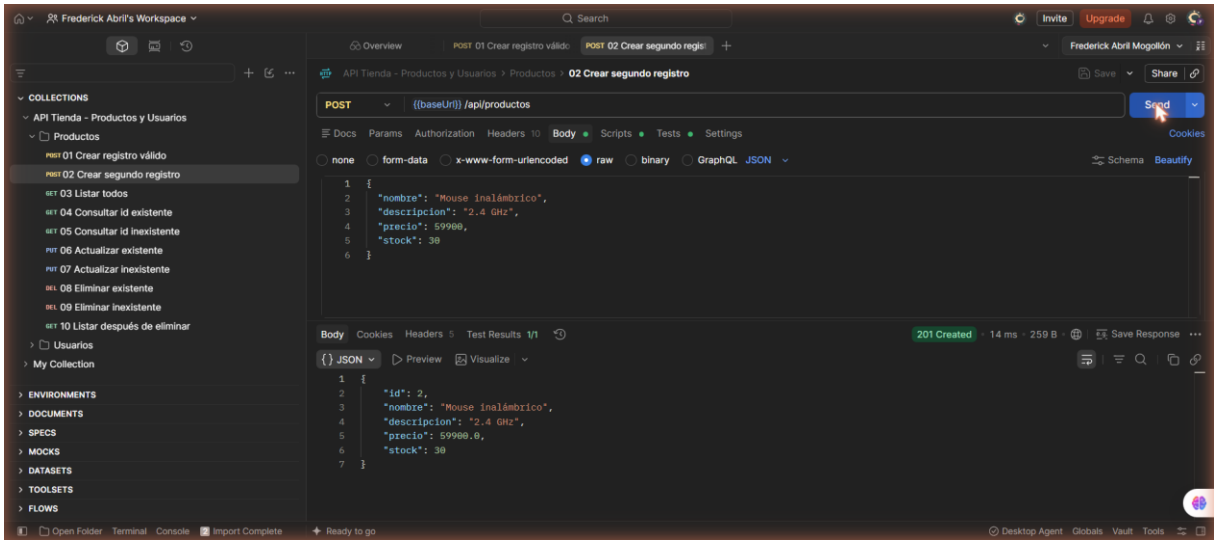


Figura 8. *Caso 2 – Crear segundo producto.*

Se crea el mouse inalámbrico y la API responde 201 con id 2.

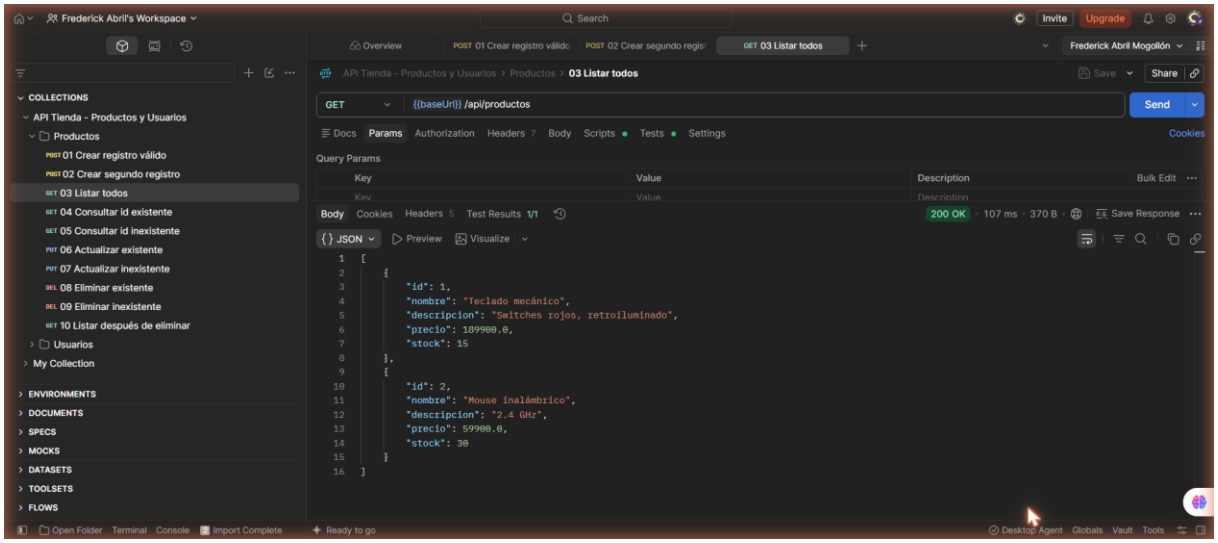


Figura 9. *Caso 3 – Listar productos.*

GET /api/productos responde 200 con un arreglo de dos elementos.

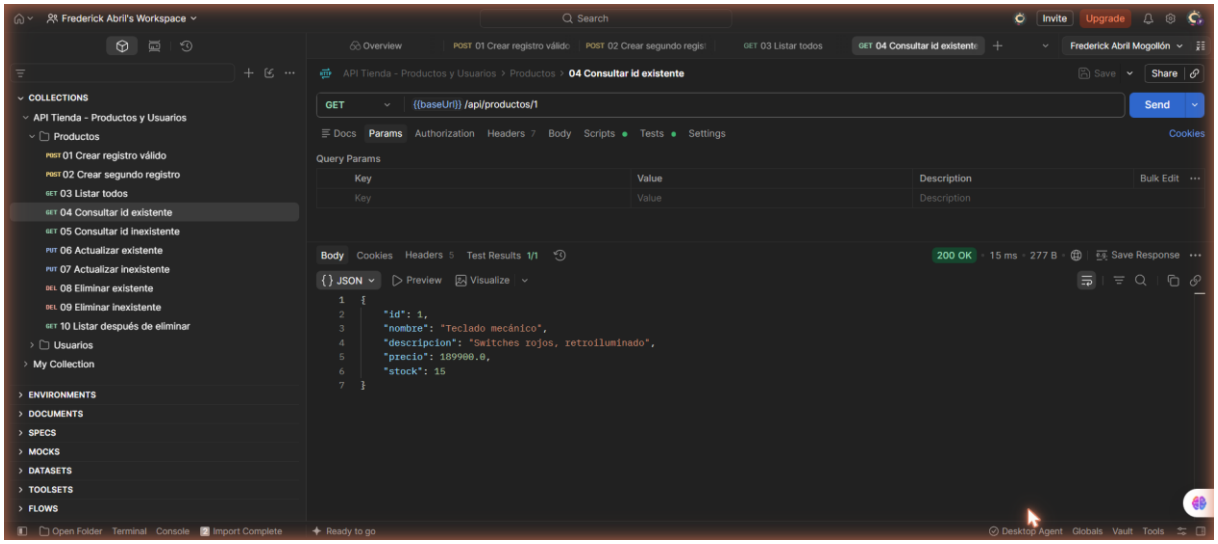


Figura 10. *Caso 4 – Consultar producto existente.*

GET /api/productos/1 responde 200 con los datos del teclado.

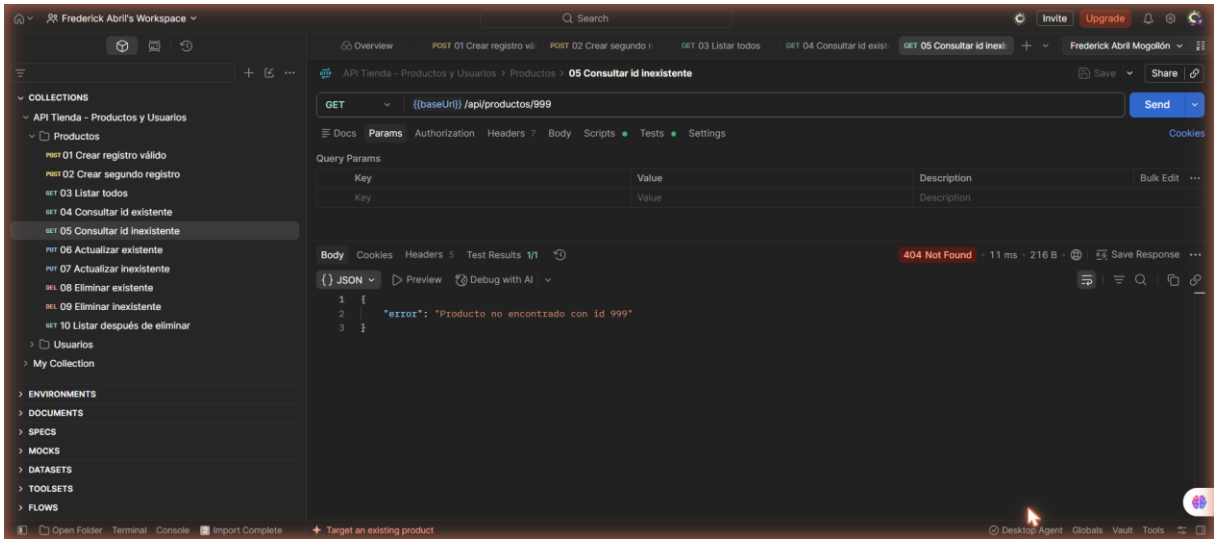


Figura 11. *Caso 5 – Consultar producto inexistente.*

GET /api/productos/999 responde 404 con el mensaje «Producto no encontrado con id 999».

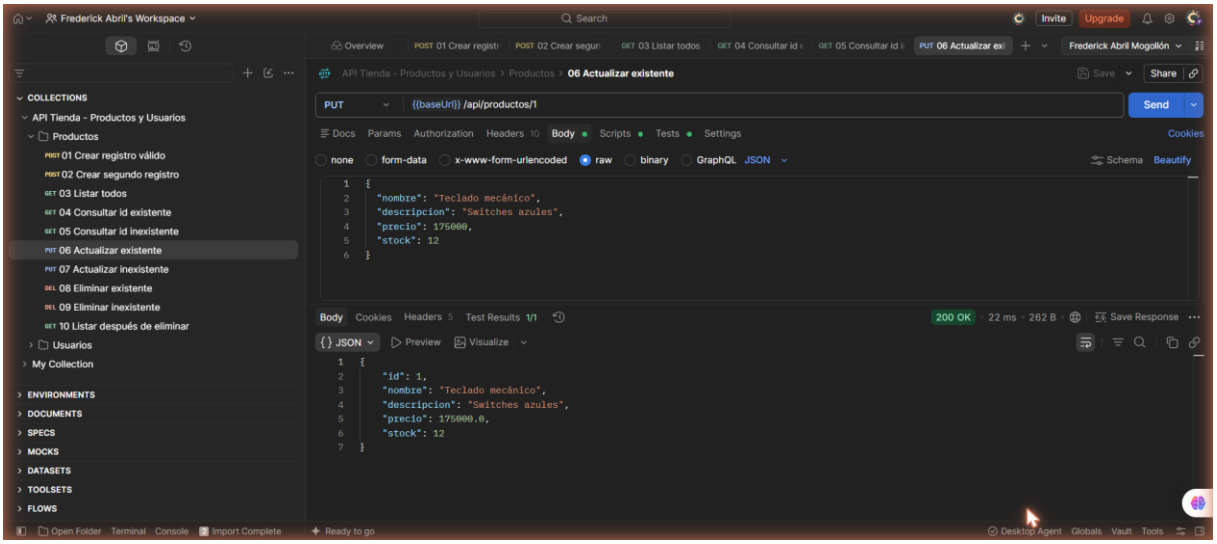


Figura 12. *Caso 6 – Actualizar producto existente.*

PUT /api/productos/1 cambia la descripción a «Switches azules», el precio a 175000 y el stock a 12; la respuesta 200 muestra los datos nuevos.

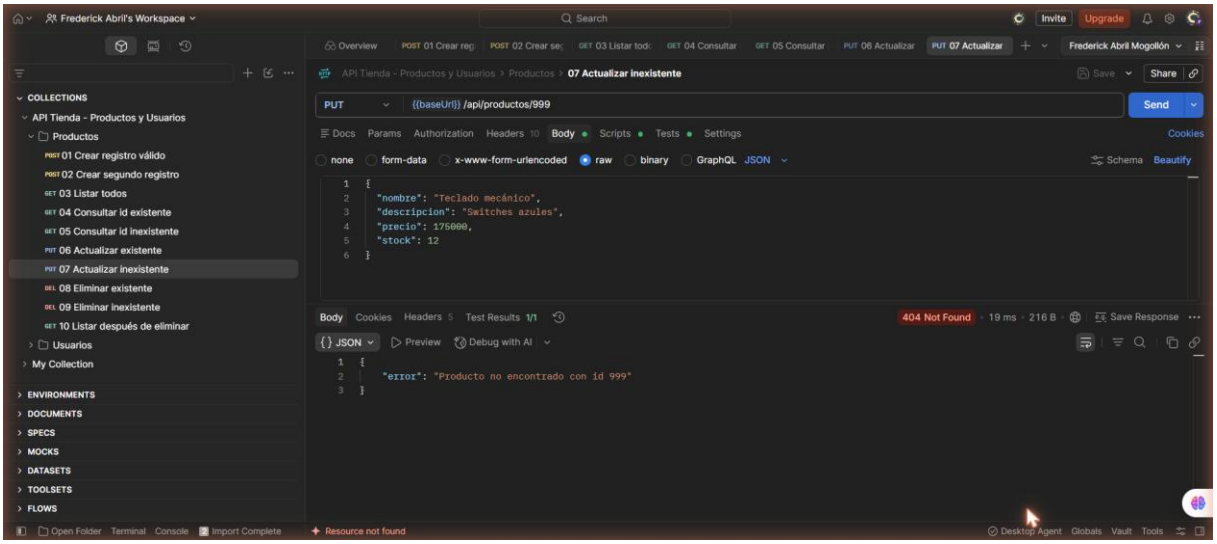


Figura 13. *Caso 7 – Actualizar producto inexistente.*

PUT /api/productos/999 responde 404 con el mensaje de error.

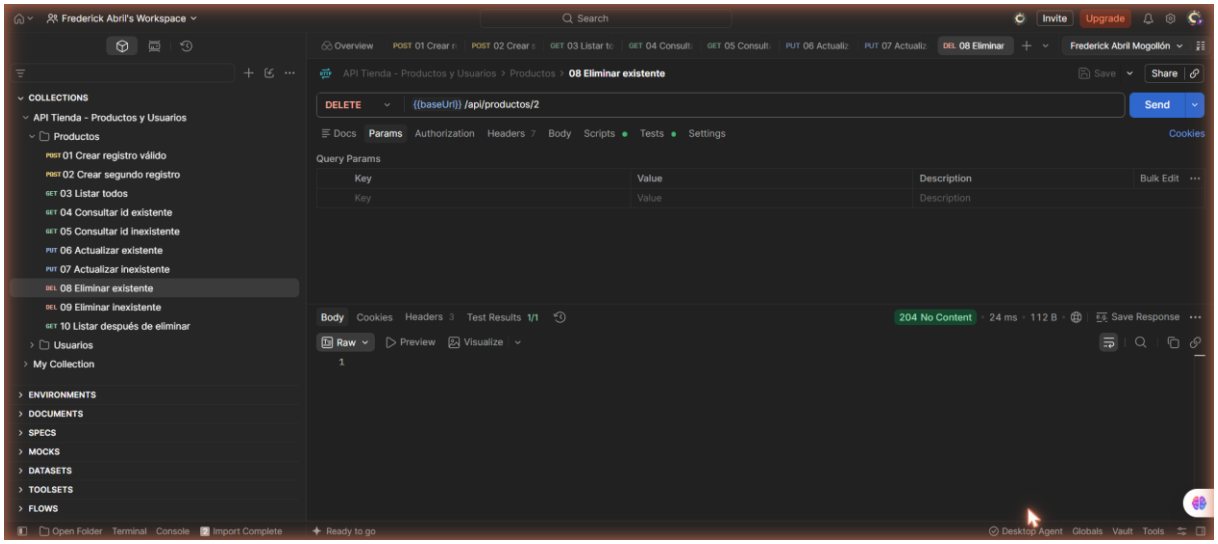


Figura 14. *Caso 8 – Eliminar producto existente.*

DELETE /api/productos/2 responde 204 No Content, sin cuerpo.

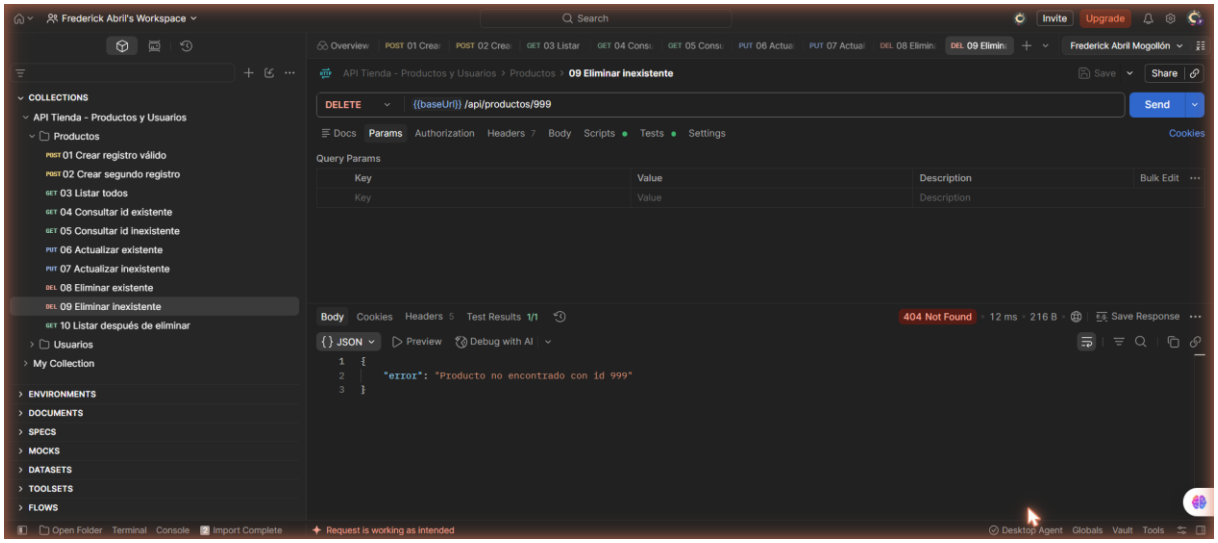


Figura 15. *Caso 9 – Eliminar producto inexistente.*

DELETE /api/productos/999 responde 404.

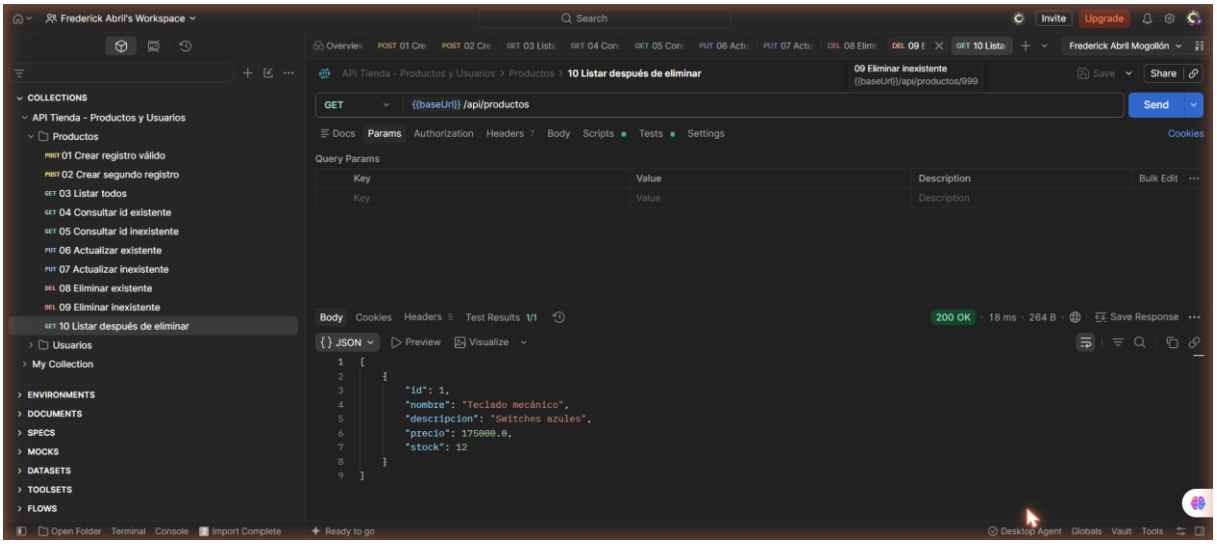


Figura 16. *Caso 10 – Listar después de eliminar.*

GET /api/productos responde 200 y ahora solo aparece el producto con id 1.

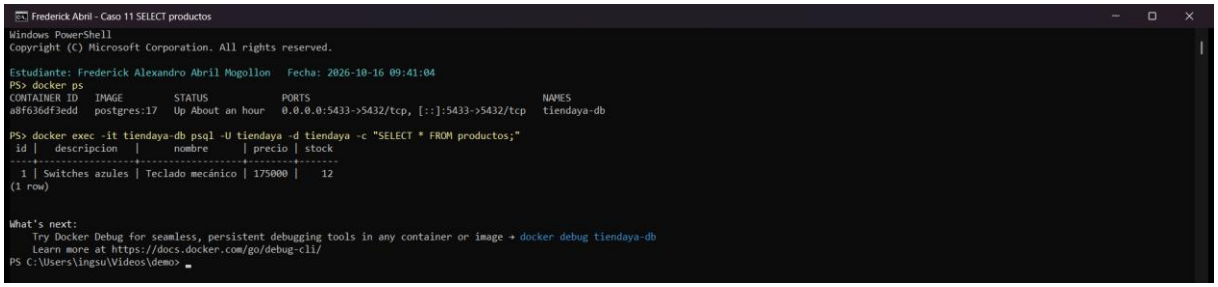


Figura 17. *Caso 11 – Verificación en la base de datos.*

docker ps muestra el contenedor tiendaya-db activo y SELECT * FROM productos devuelve el mismo registro que entrega la API después de las pruebas.

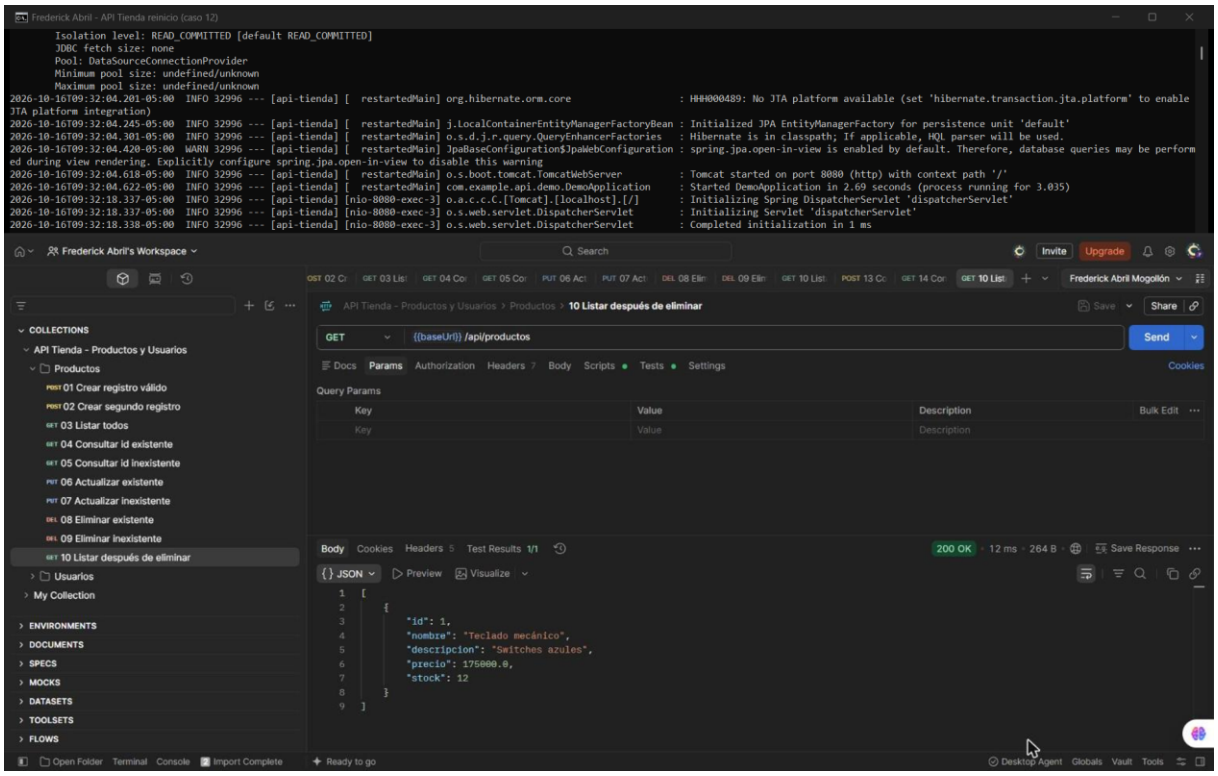


Figura 18. *Caso 12 – Persistencia de productos.*

Detuve la API y la volví a arrancar (parte superior, Started DemoApplication a las 09:32:04). Luego GET /api/productos devolvió el mismo producto, así que los datos se conservaron.

Evidencias del módulo de usuarios

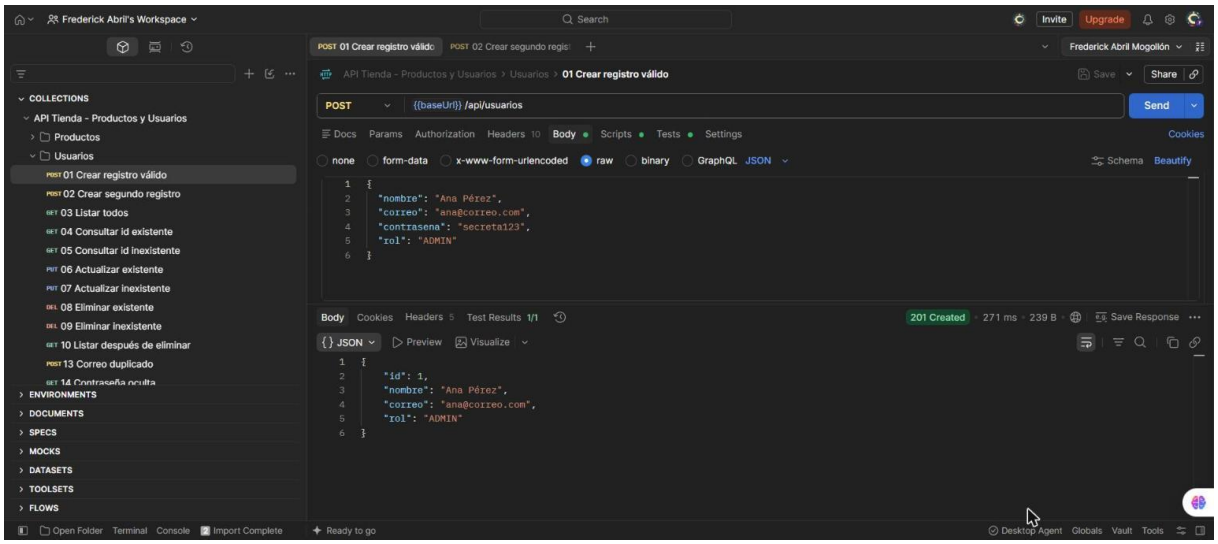


Figura 19. *Caso 1 – Crear usuario válido.*

POST /api/usuarios con Ana Pérez responde 201 con id 1. La contraseña se envía en la petición pero no aparece en la respuesta.

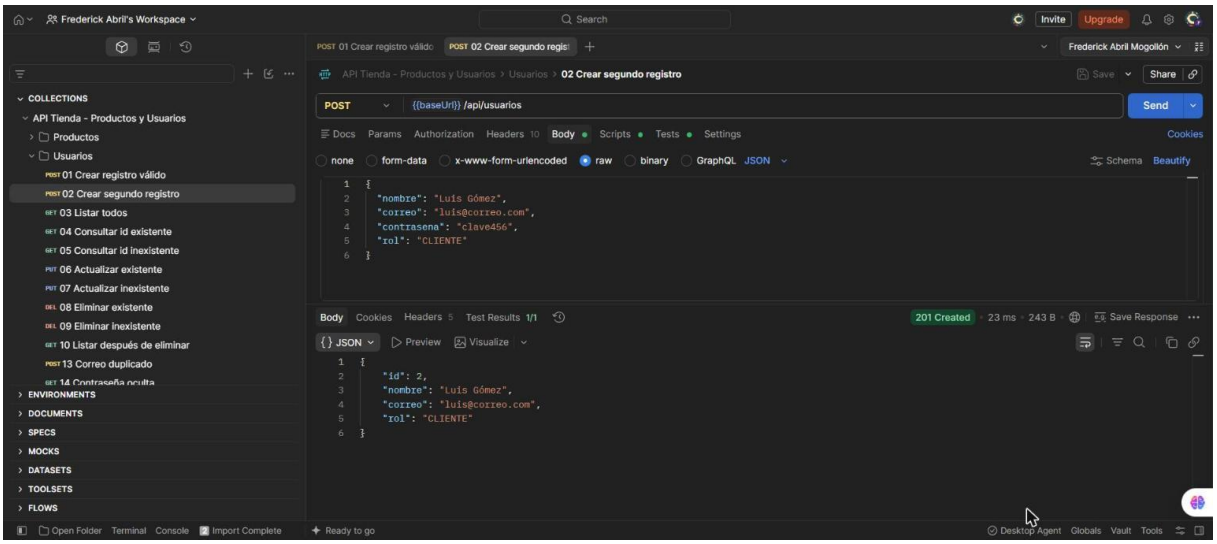


Figura 20. *Caso 2 – Crear segundo usuario.*

Se crea a Luis Gómez y la API responde 201 con id 2.

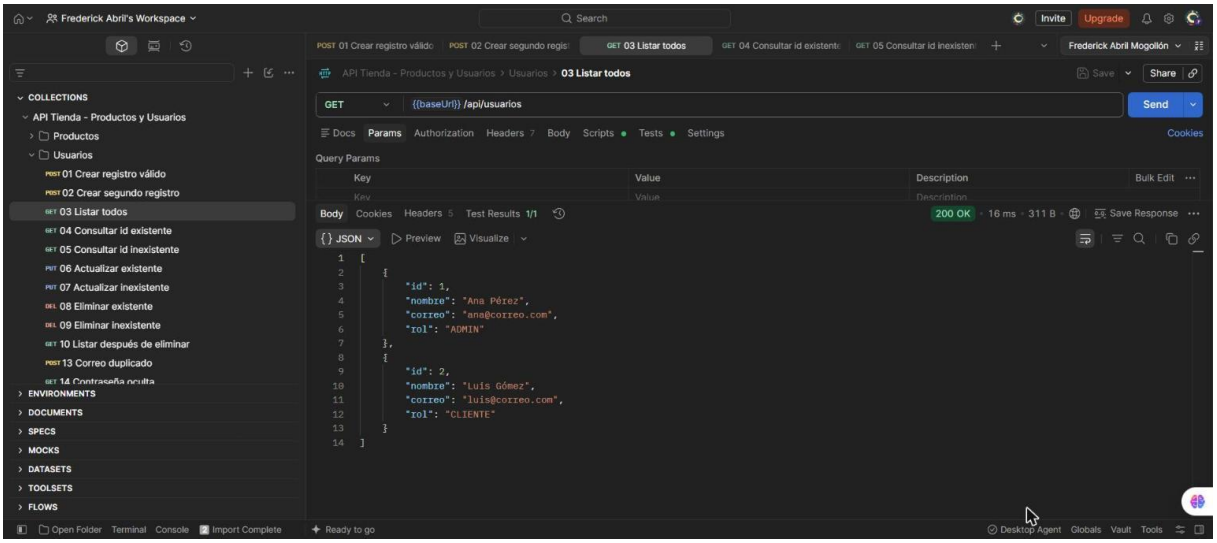


Figura 21. *Caso 3 – Listar usuarios.*

GET /api/usuarios responde 200 con los dos usuarios, ninguno con el campo contrasena.

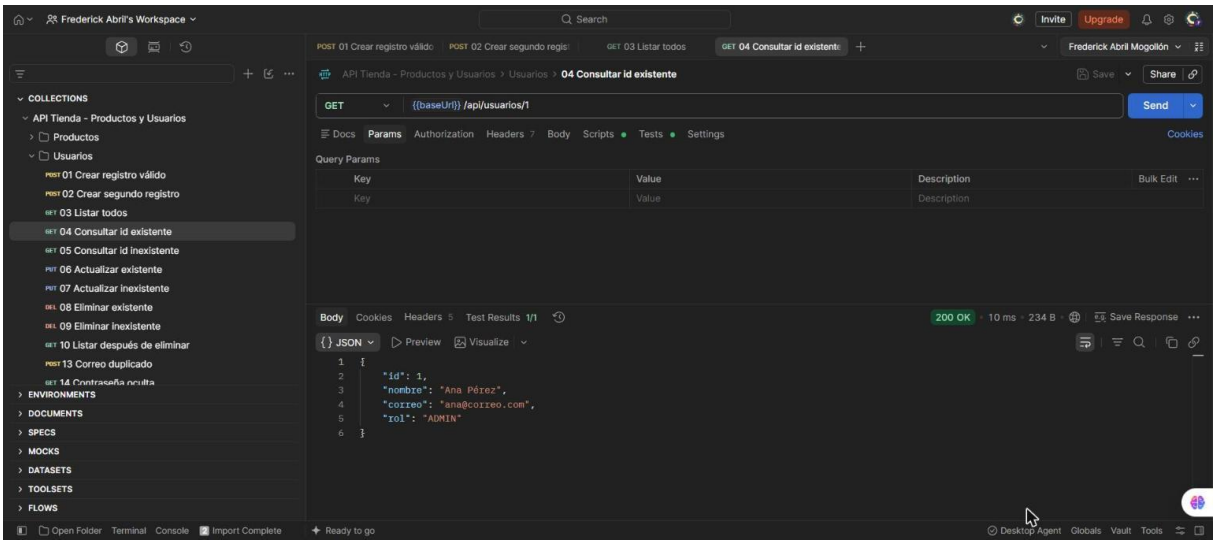


Figura 22. *Caso 4 – Consultar usuario existente.*

GET /api/usuarios/1 responde 200 con los datos de Ana Pérez.

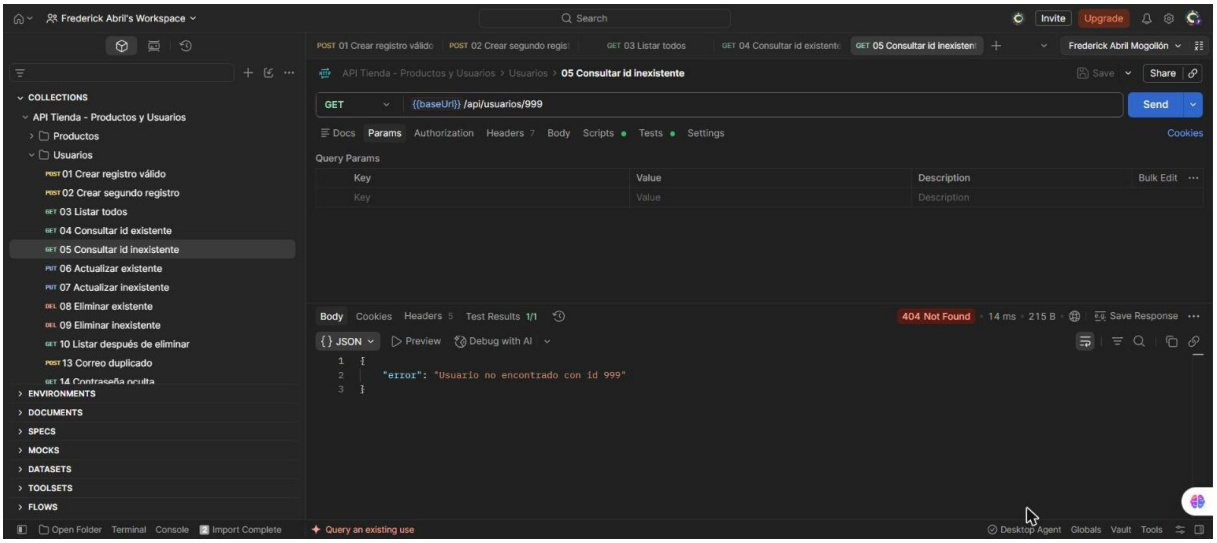


Figura 23. *Caso 5 – Consultar usuario inexistente.*

GET /api/usuarios/999 responde 404 con «Usuario no encontrado con id 999».

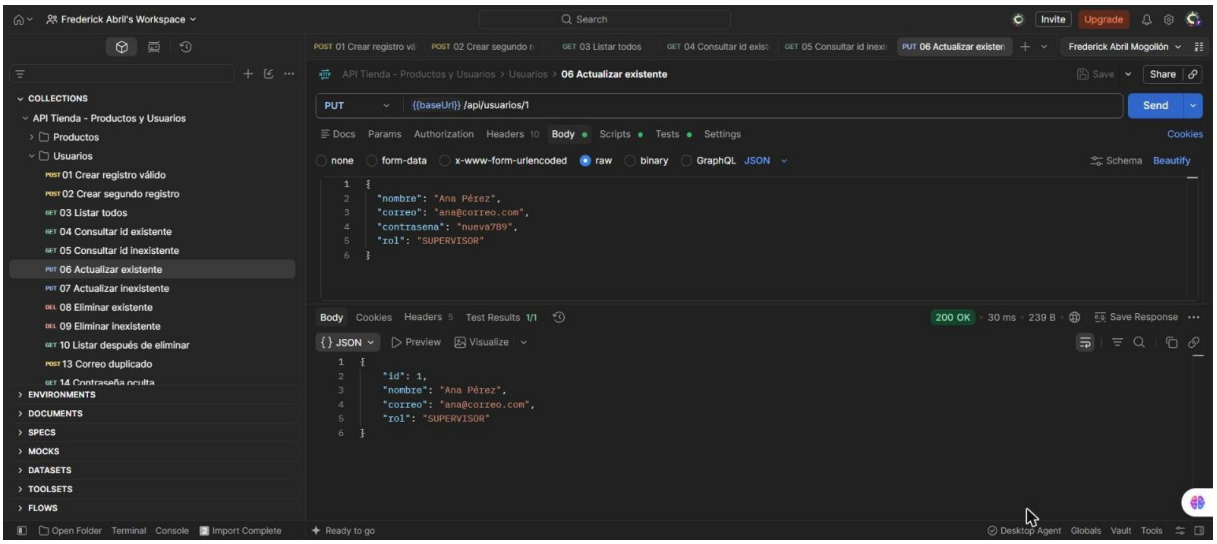


Figura 24. *Caso 6 – Actualizar usuario existente.*

PUT /api/usuarios/1 cambia la contraseña y el rol a SUPERVISOR; la respuesta 200 muestra el rol nuevo.

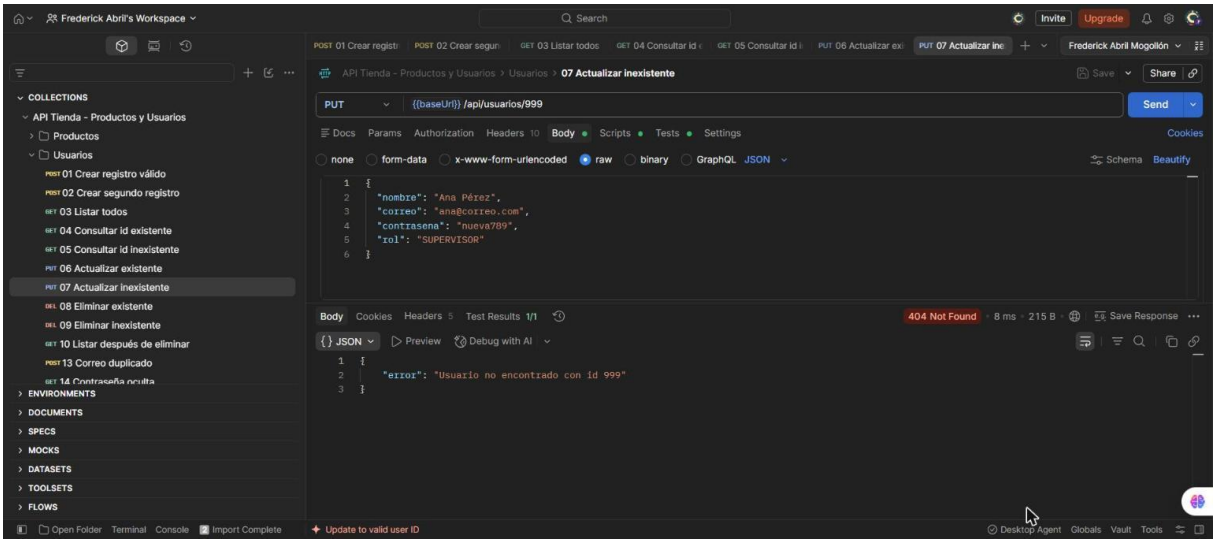


Figura 25. *Caso 7 – Actualizar usuario inexistente.*

PUT /api/usuarios/999 responde 404.

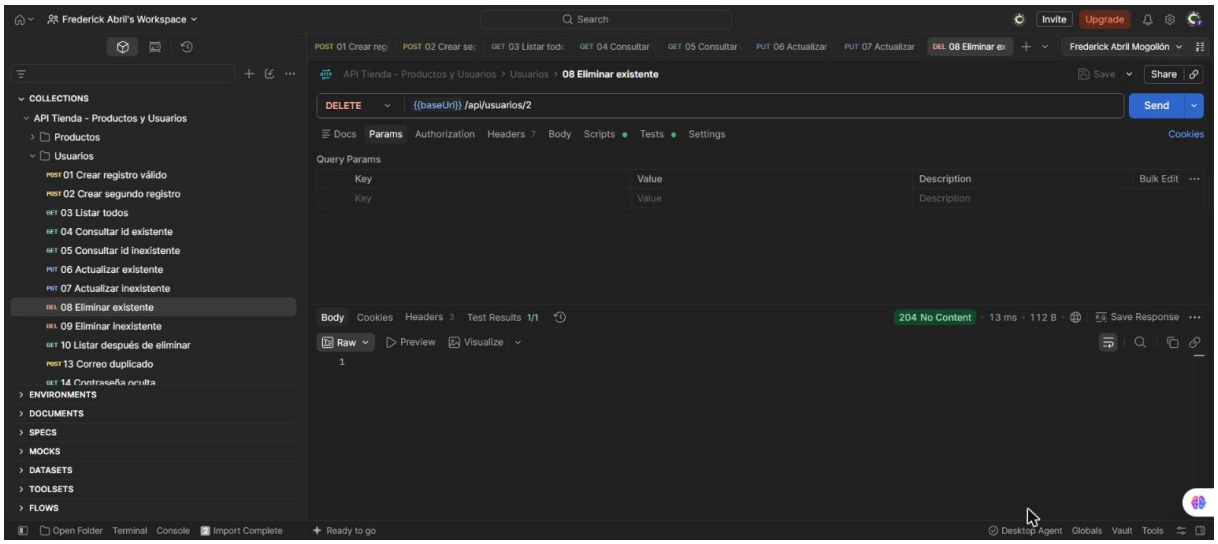


Figura 26. *Caso 8 – Eliminar usuario existente.*

DELETE /api/usuarios/2 responde 204 No Content.

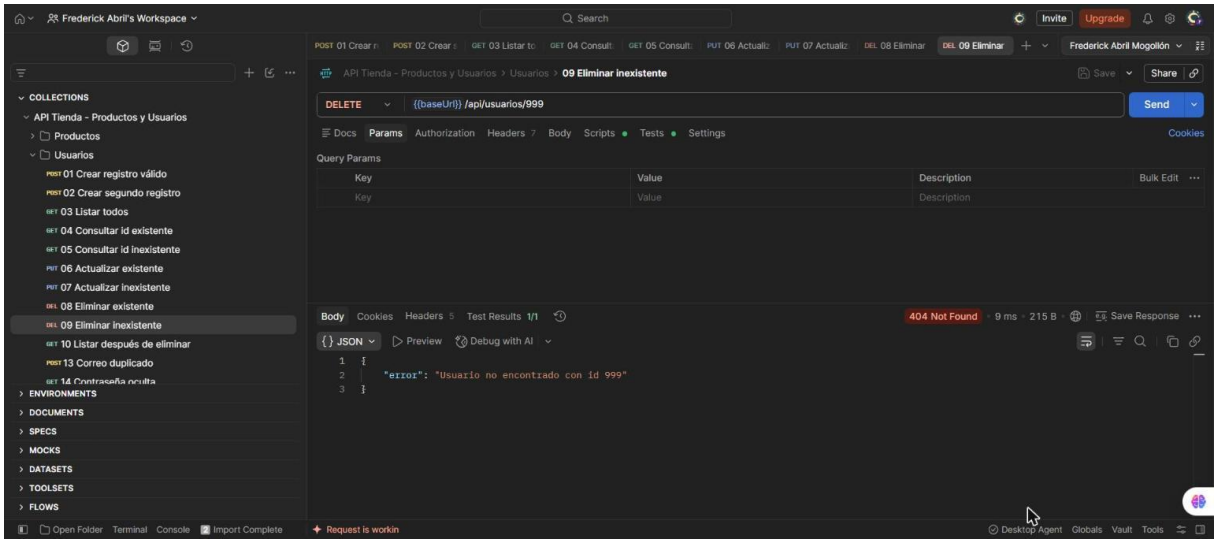


Figura 27. *Caso 9 – Eliminar usuario inexistente.*

DELETE /api/usuarios/999 responde 404.

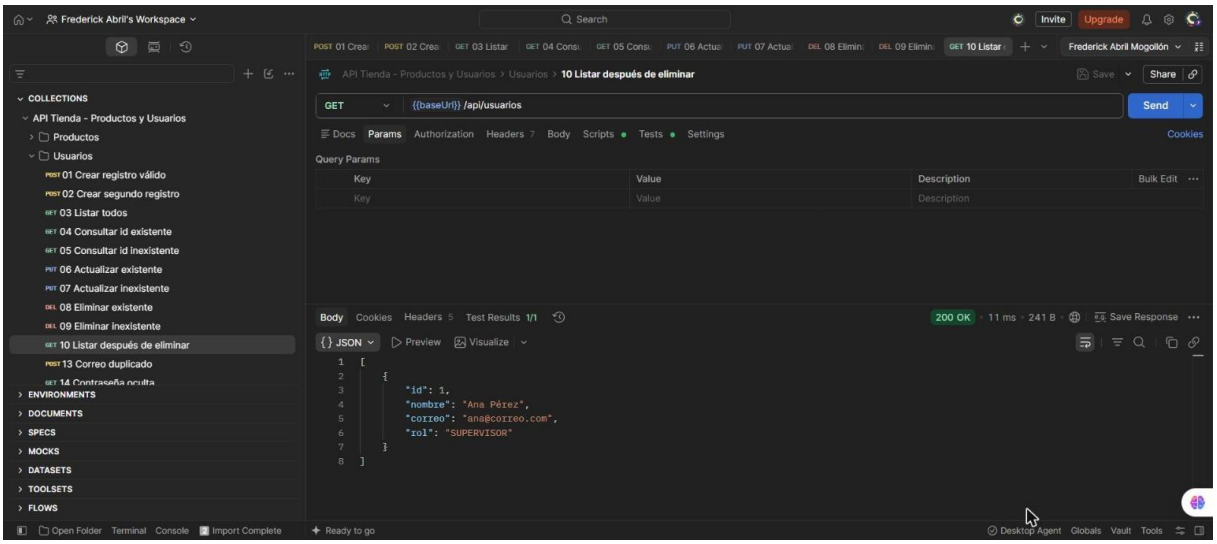


Figura 28. *Caso 10 – Listar usuarios después de eliminar.*

GET /api/usuarios responde 200 y solo queda Ana Pérez.



Figura 29. *Caso 11 – Verificación en la base de datos.*

SELECT * FROM usuarios coincide con la API. Aquí se ve que la contraseña se guarda en texto plano; la API no la devuelve, pero en un sistema real habría que cifrarla, por ejemplo con BCrypt.

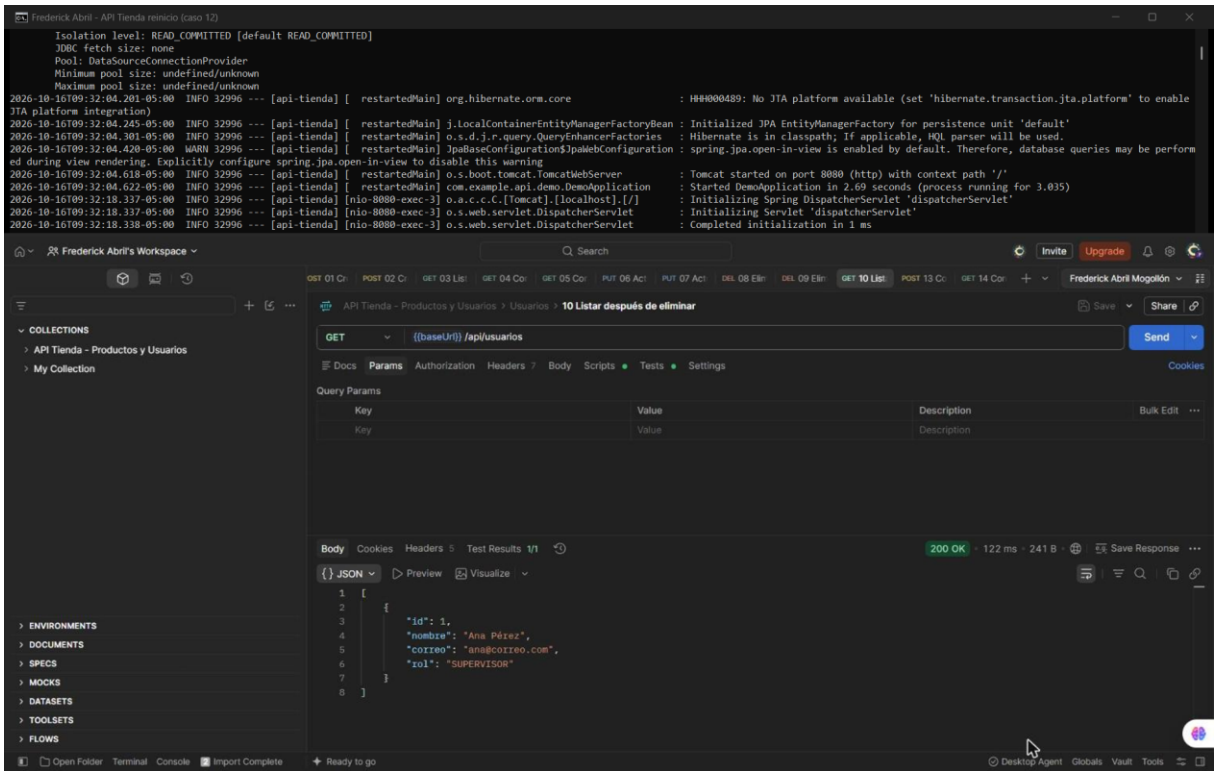


Figura 30. *Caso 12 – Persistencia de usuarios.*

Después del mismo reinicio de la API, GET /api/usuarios devuelve a Ana Pérez con sus datos actualizados.

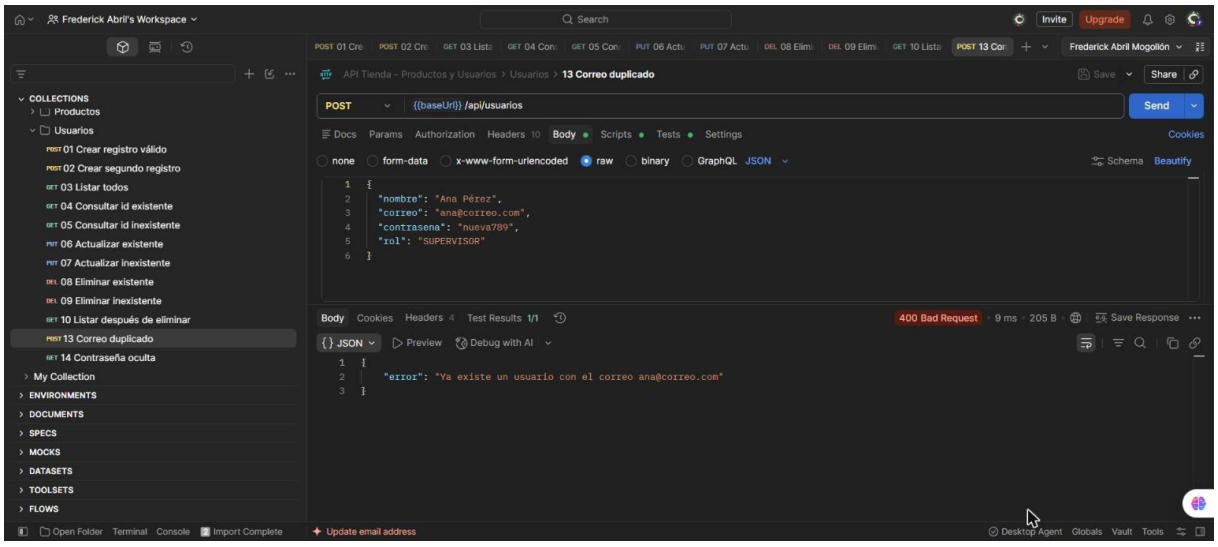


Figura 31. *Caso 13 – Correo duplicado.*

Crear otro usuario con ana@correo.com responde 400 con el mensaje «Ya existe un usuario con el correo ana@correo.com».

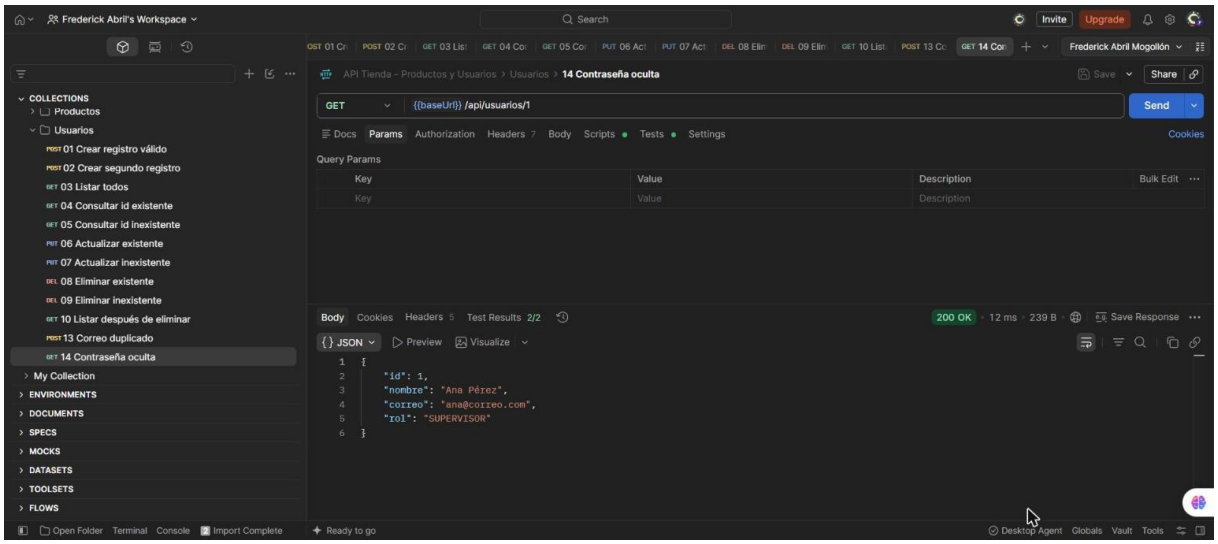


Figura 32. *Caso 14 – Contraseña oculta.*

GET /api/usuarios/1 responde 200 y el JSON solo trae id, nombre, correo y rol; el campo contrasena no aparece.

9. Conclusiones

Con este taller aprendí que muchos errores no están en el código sino en el entorno. La mayor dificultad fue el error de autenticación con PostgreSQL: la contraseña estaba bien, pero la aplicación se conectaba al PostgreSQL instalado en Windows y no al contenedor. Lo resolví cambiando el puerto del contenedor a 5433 y dejando los datos de conexión en variables de entorno, para poder cambiarlos sin tocar el código.

También entendí la utilidad de Lombok y de la arquitectura por capas. Las clases quedaron más cortas y fáciles de leer, y cuando tuve que agregar la regla del correo único solo modifiqué el servicio y el repositorio de usuarios. Otra dificultad fue que la prueba contextLoads dependía de la base de datos; la resolví usando H2 en memoria solo para las pruebas.

Por último, ejecutar cada caso en Postman y tener además las pruebas automáticas me dio confianza en que la API responde con los códigos correctos. La prueba de persistencia confirmó que los datos quedan guardados en PostgreSQL aunque la aplicación se detenga. Como mejora pendiente queda cifrar las contraseñas antes de guardarlas.

10. Anexos

Repositorio del proyecto en GitHub: https://github.com/Fredpuff/api-tienda-springboot

Colección de Postman exportada: postman/API_Tienda.postman_collection.json, con las carpetas Productos y Usuarios. El environment usado en las pruebas está en postman/Frederick_Abril_Mogollon.postman_environment.json.

Las capturas originales en tamaño completo están en la carpeta evidencias del repositorio, numeradas en el mismo orden de las figuras de este informe.